

Here are guidelines for writing Java programs in this course.

Spaces

Put spaces on both sides of arithmetic operators.

```
// Incorrect // Correct
a=(b*4+7.5)/c; a = (b * 4 + 7.5) / c;
```

The keywords `if`, `else`, `for`, `do`, and `while` are always followed by a space. Put a space between a closing parenthesis and an opening brace. When a closing brace is on the same line as an `else`, follow it with a space.

```
// Incorrect // Correct
if(a < b){    if (a < b) {
    for(int i = 0; i < 10; i++){
        j += i * a;
    }
} else{       for (int i = 0; i < 10; i++) {
    j = b * 9;   j += i * a;
}              }
               } else {
               j = b * 9;
               }
```

Do *not* put spaces after an opening parenthesis or before a closing parenthesis:

```
// Incorrect // Correct
for ( int i = 0; i < 5; i++ ) {    for (int i = 0; i < 5; i++) {
    sum = ( sum + i ) * Math.sqrt( 2 );    sum = (sum + i) * Math.sqrt(2);
}                                  }
```

Braces and Indenting

There are several ways of placing braces in your Java source code. One thing they all have in common: closing braces are always the first non-whitespace character on a line. They are *never* placed at the end of a line:

```
// Incorrect
if (a < b) {
    result = 22; }
else
    result = 99; }
```

Any of these are correct; the book we are currently using uses the first style:

```
// Correct // Correct // Correct
if (a < b) {    if (a < b)    if (a < b) {
    result = 22; {    result = 22;    result = 22;
} else {        } else {      } else {
    result = 99; }    result = 99;    result = 99;
}                {
                result = 99;
                }
```

Always enclose the body of an `if` clause, `else` clause, or loop in braces, even when there is only one statement in the body:

```
// Incorrect // Correct
if (a < b)    if (a < b) {
    result = 22;
else        } else {
    result = 99;
}          result = 99;
}          }

for (int i = 0; i < 7; i++)
    result += i * i;

for (int i = 0; i < 7; i++) {
    result += i * i;
}
```

You may indent either two or four spaces, and you may use the TAB character instead of spaces. You have to be correct and consistent in your indenting and placement of braces. Note how the `if-else-if` chain is aligned in the correct example:

```
//Incorrect //Correct
if (a < b) {    if (a < b) {
    if (c >= d) {    if (c >= d) {
        result = 2;        result = 2;
    } else if (e == f)    } else if (e == f) {
{        result = 99;    result = 99;
    }          }
}          }
```

Names and Declarations

Variable names begin with lower case letters. If you have a multi-word variable name, prefer `camelCase` to `snake_case`. If you have a `final` (constant), use all upper case for its name, and, if it is a multi-word name, use `SNAKE_CASE`. Names of Java classes begin with a upper case letter:

```
// Incorrect // Correct
class example {    class Example {
    int Counter;        int counter;
    double unitprice;    double unit_price; // acceptable
    final double saletaxrate = 0.075;    double unitPrice; // preferred
    // ...        final double SALES_TAX_RATE = 0.075;
}                // ..
}
```

Here are guidelines for writing Java programs in this course.

Spaces

Put spaces on both sides of arithmetic operators.

```
// Incorrect // Correct
a=(b*4+7.5)/c; a = (b * 4 + 7.5) / c;
```

The keywords `if`, `else`, `for`, `do`, and `while` are always followed by a space. Put a space between a closing parenthesis and an opening brace. When a closing brace is on the same line as an `else`, follow it with a space.

```
// Incorrect // Correct
if(a < b){ if (a < b) {
    for(int i = 0; i < 10; i++){
        j += i * a;
    }
} else{ } else {
    j = b * 9;
} }
```

Do *not* put spaces after an opening parenthesis or before a closing parenthesis:

```
// Incorrect // Correct
for ( int i = 0; i < 5; i++ ) { for (int i = 0; i < 5; i++) {
    sum = ( sum + i ) * Math.sqrt( 2
); sum = (sum + i) *
} Math.sqrt(2);
}
```

Braces and Indenting

There are several ways of placing braces in your Java source code. One thing they all have in common: closing braces are always the first non-whitespace character on a line. They are *never* placed at the end of a line:

```
// Incorrect
if (a < b) {
```

```
    result = 22; }
else
    result = 99; }
```

Any of these are correct; the book we are currently using uses the first style:

```
// Correct
if (a < b) {
    result = 22;
} else {
    result = 99;
}

// Correct
if (a < b) {
    result = 22;
} else {
    {
        result = 99;
    }
}

// Correct
if (a < b) {
    result = 22;
} else {
    result = 99;
}
```

Always enclose the body of an `if` clause, `else` clause, or loop in braces, even when there is only one statement in the body:

```
// Incorrect
if (a < b)
    result = 22;
else
    result = 99;

for (int i = 0; i < 7; i++)
    result += i * i;

// Correct
if (a < b) {
    result = 22;
} else {
    result = 99;
}

for (int i = 0; i < 7; i++) {
    result += i * i;
}
```

You may indent either two or four spaces, and you may use the TAB character instead of spaces. You have to be correct and consistent in your indenting and placement of braces. Note how the `if-else-if` chain is aligned in the correct example:

```
//Incorrect
if (a < b) {
    if (c >= d) {
        result = 2;
    } else if (e == f)
    {
        result = 99;
    }
}

//Correct
if (a < b) {
    if (c >= d) {
        result = 2;
    } else if (e == f) {
        result = 99;
    }
}
```

Names and Declarations

Variable names begin with lower case letters. If you have a multi-word variable name, prefer `camelCase` to `snake_case`. If you have a `final` (constant), use all upper case for its name, and, if it is a multi-word name, use `SNAKE_CASE`. Names of Java classes begin with a upper case letter:

```
// Incorrect
class example {
    int Counter;
    double unitprice;
    final double salestaxrate =
0.075;
    // ...
}
```

```
// Correct
class Example {
    int counter;
    double unit_price; // acceptable
    double unitPrice;  // preferred
    final double SALES_TAX_RATE =
0.075;
    // ..
}
```

Use meaningful variable names.

// Incorrect	// Correct
int y;	int years;
double p;	double price;
double t;	double tax;

(The exceptions are for well-known formulas such as the quadratic formula, where you expect to see coefficients `a`, `b`, and `c`. Loop counters are often written as in algebraic subscripts: `i` and `j`, but if you can make a more meaningful name—especially in nested loops, that will improve readability.)

Put only one declaration per line:

// Incorrect	// Correct
double cut, clarity, caratWeight, color;	double cut;
	double clarity;
	double caratWeight;
	double color;

Long Lines

Limit line length to 75, no more than 79, characters. Many IDEs will let you display a “line length marker” and set its limit. They might also show the current column number in the status bar at the bottom of the editor window.

Prefer multiple `println()` calls to one `print()` or `println()` with multiple `\n`:

```
// difficult to read and edit
System.out.print("Weight: " + weight + "\nVolume: " + volume +
"\nSurface Area: " + area + "\n");
```

```
// better
System.out.println("Weight: " + weight);
System.out.println("Volume: " + volume);
System.out.println("Surface Area: " + area);
```

If you break a formula across lines, you can break with the operator at the end of a line or beginning of the next line. Just be consistent.

```
// Either of these is correct
double first = (weight * volume) / (length + width / 2 *
    height * height);
double second = (weight * volume) / (length + width / 2
    * height * height);
```

Comments

The code tells you *what* the program does. Comments should tell you *why* or *how*. (The book will often have “what” comments because the book has to explain what a concept is.) Comments should precede the line instead of after the line. Comment blocks of code and not individual lines.

```
// "what" comment
month += 1; // add one to
month
```

```
// "why" comment
/* System returns 0-11 for January-
December,
 * but people expect months to be 1-12.
 */
month += 1;
```

The exception to this guideline is the comments at the start of your program. These must have your name and a description of the purpose of the program. The description must be clear enough that someone who has not read the assignment will understand what the program does. These comments are incorrect:

```
/*
 * Assignment 3
 * J. Fulano - Oct 4, 2525
 */
```

```
/*
 * Program uses if statements
 * and while loops.
 * J. Fulano - Oct 4, 2525
 */
```

This one is correct (Note the use of [Javadoc comment formatting](#))

```
/**
 * Ask user to enter a positive
integer, and print out whether
 * that integer is prime or not.
 * Program does this repeatedly
until user enters zero as input;
 * negative integers give an error
message.
 * @author J. Fulano
 */
```

Loops

Avoid `break` and `continue` whenever possible; instead, carefully think through the loop condition. Do *not* use `while (true)` with `break` to substitute for thinking through the correct condition!

Consider the following code to read into an array until it is full or the user enters zero:

```
// Very bad style
final int MAX = 4;
int [] ages = new
int[MAX];
int value = -1;
int count = 0;

while (true) {

System.out.print("Enter
age or 0 to quit: ");
    value =
input.nextInt();
    if (value == 0) {
        break;
    }
}
```

```
// better
final int MAX = 4;
int [] ages = new
int[MAX];
int value = -1;
int count = 0;

while (value != 0) {

System.out.print("Enter
age or 0 to quit: ");
    value =
input.nextInt();
    if (value != 0) {
        ages[count] =
value;
    }
}
```

```
// best
final int MAX = 4;
int [] ages = new
int[MAX];
int value = -1;
int count = 0;

while (count < MAX &&
value != 0) {

System.out.print("Enter
age or 0 to quit: ");
    value =
input.nextInt();
    if (value != 0) {
        ages[count] =
value;
    }
}
```

<pre> ages[count] = value; count++; if (count == MAX) { break; } } </pre>	<pre> count++; if (count == MAX) { break; } } } </pre>	<pre> value; count++; } } </pre>
---	--	--

Methods

Do not put a space between the method name and the parameter list.

<pre> // Incorrect static double cube (double value) { return value * value * value; } // ... double result = cube (12.0); </pre>	<pre> // Correct static double cube(double value) { return value * value * value; } // ... double result = cube(12.0); </pre>
---	---

It is a good idea to put comments before a method that describes the method's purpose, inputs, and return value (if any). Please use [Javadoc comment format](#):

```

/**
 * Calculate monthly payment on a loan, given:
 * @param principal amount of the loan
 * @param interestRate annual percentage rate as a decimal (0.075 for
7.5%)
 * @param years number of years of the loan
 * @returns the monthly payment amount of the loan
 */
static double payment(double principal, double interestRate, int years)
{
    //...
}

```

Input

Input prompts must give the user an idea of what kind of input is expected. The input cursor must appear on the same line as the prompt. It is usually good to put a space after the prompt to give the cursor some "breathing

room.” In the following examples, the black square shows where the cursor appears.

// Incorrect	// OK	// Correct
<code>System.out.println("Enter data:");</code>	<code>System.out.print("Enter height in cm:");</code>	<code>System.out.print("Enter height in cm: ");</code>
Enter data: ■	Enter height in cm:■	Enter height in cm: ■

Note: when asking for currency as input, the cursor can appear directly after the currency symbol:

```
System.out.print("Enter item price: $");
```

```
Enter item price: $■
```

Output

When lining up output, prefer spaces and specific formatting over tabs:

// OK	// Better
<code>System.out.printf("Height (cm):\t\t%.2f\n", height);</code>	<code>System.out.printf("Height (cm): %6.2f\n", height);</code>
<code>System.out.printf("Weight (kg):\t\t%.2f\n", weight);</code>	<code>System.out.printf("Weight (kg): %6.2f\n", weight);</code>
<code>System.out.printf("Blood alcohol:\t%.3f%%\n", bac);</code>	<code>System.out.printf("Blood alcohol: %7.3f%%\n", bac);</code>
Height (cm): 160.30	Height (cm): 160.30
Weight (kg): 50.40	Weight (kg): 50.40
Blood alcohol: 0.032%	Blood alcohol: 0.032%